Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	Shravan P M	
Roll Number	24110026	
Registration Number	24011101101	
Class / Section	B.Tech AI & DS - B	

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete source code for this experiment has been uploaded to a GitHub repository for reference. The repository is periodically updated with code refinements, additional implementations, and detailed insights gained through successive iterations of the experiment.

GitHub Repository:

`https://github.com/shravW/Deep-Learning-Lab-CS3807-SEM-5/tree/main/Ex_1_DL`

7. Experimental Results

Dataset Statistics

The Banknote Authentication dataset comprises 1372 instances and 4 input features and has 1 class variable. Each record in the dataset has 4 numerical input variables, and the class variable is binary. All variables are stored as float64, and the class is stored as int64. The data does not have missing values. The data description is a complete set, and no data cleaning was needed.

Below are statistical descriptions of the data.

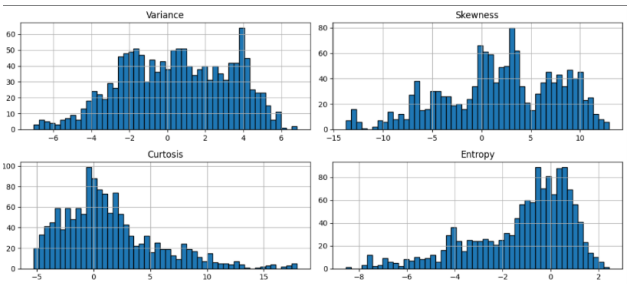
Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
Mean	0.433735	1.922353	1.397627	-1.191657	0.444606
Std	2.842763	5.869047	4.310030	2.101013	0.497103
Min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
Max	6.824800	12.951600	17.927400	2.449500	1.000000

Other Observations

- The given data set consists of 1372 samples, of which four are input features and the fifth is the target class.
- All of the data set's features have no missing data entries. Secondly, the data set was normalized using StandardScaler before proceeding to model training.
- An 80:20 ratio was used to split the data set into train and test subsets, resulting in 1097 and 275 observations respectively. A Single Layer Perceptron was trained with a learning rate of 0.01 and for 50 epochs.
- The model learned the decision boundary and was evaluated using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

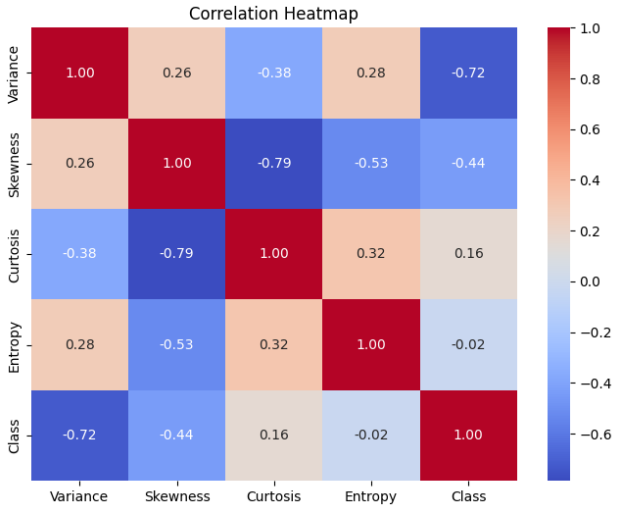
8. Mandatory Plots with Interpretation

Feature Histograms



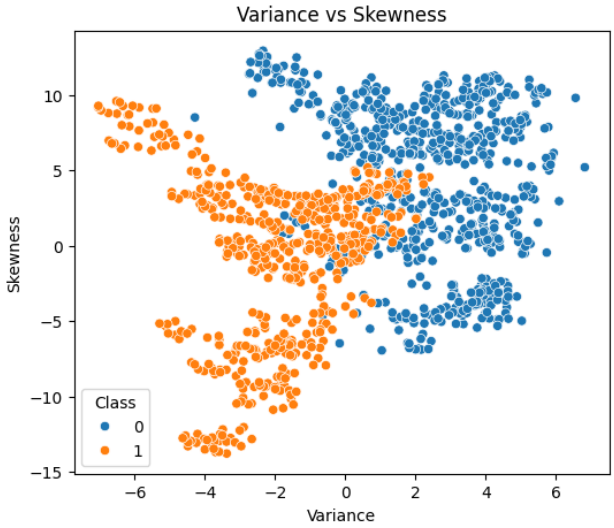
The histograms illustrate the distribution of the four numerical features in the Banknote Authentication dataset. *Variance* and *Skewness* exhibit approximately symmetric distributions, with a higher concentration of samples around their central values. In contrast, *Curtosis* displays a left-tailed (negatively skewed) distribution, whereas *Entropy* exhibits a right-tailed (positively skewed) distribution. Both *Curtosis* and *Entropy* also contain noticeable outliers, as indicated by their long tails and isolated observations. The varying distributions indicate that each feature possesses distinct statistical characteristics, allowing them to collectively contribute towards distinguishing authentic and forged banknotes.

Correlation Heatmap



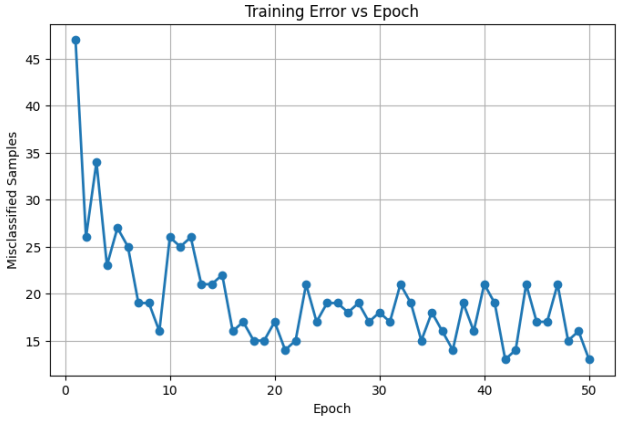
The correlation heatmap illustrates the linear relationship between the input features and the target class. Among the four input features, *Variance* exhibits the strongest correlation with the target class, having a correlation coefficient of approximately -0.72, indicating that it is the most informative feature for classification. *Curtosis* is the only feature that shows a positive correlation with the target class (0.16), while *Entropy* has an almost negligible correlation (-0.02). Among the input features, *Skewness* and *Curtosis* exhibit the strongest correlation with each other, having a coefficient of approximately -0.79. The heatmap provides a clear understanding of the relationships among the features and highlights their relevance to the binary classification task.

Scatter Plot



The scatter plot of \textit{Variance} versus \textit{Skewness} illustrates the distribution of authentic and forged banknotes based on two of the most informative features in the dataset. The two classes form distinct clusters with only a small region of overlap, indicating that these features provide good discriminatory information for binary classification. Although perfect separation is not achieved, the overall distribution suggests that the data is approximately linearly separable. This explains why the Single Layer Perceptron is able to learn an effective decision boundary and achieve high classification accuracy with only a few misclassifications.

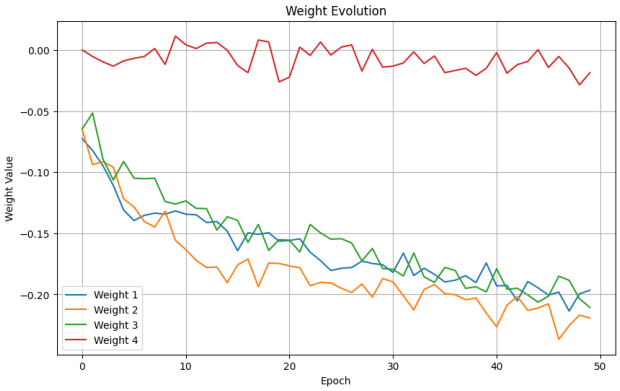
Training Error vs Epoch



The Training Error vs Epoch plot illustrates the learning behaviour of the Single Layer Perceptron during training. The number of misclassified samples decreases rapidly during the initial epochs, indicating that the model quickly learns the dominant patterns present in the dataset. After the initial reduction, the training error fluctuates within a narrow range while remaining significantly lower than its initial value. These fluctuations indicate that the dataset is not perfectly linearly sep-

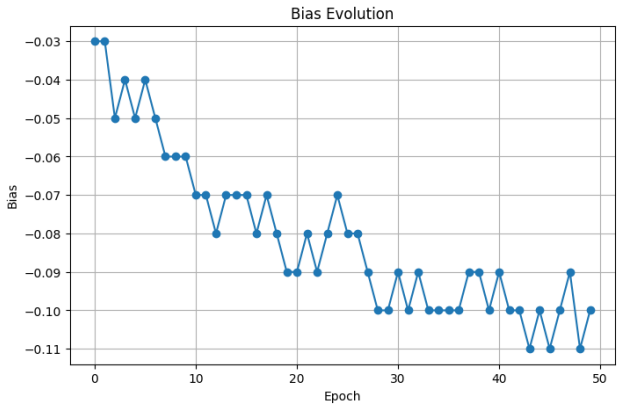
arable, preventing complete convergence of the perceptron despite achieving effective classification performance.

Weight Evolution



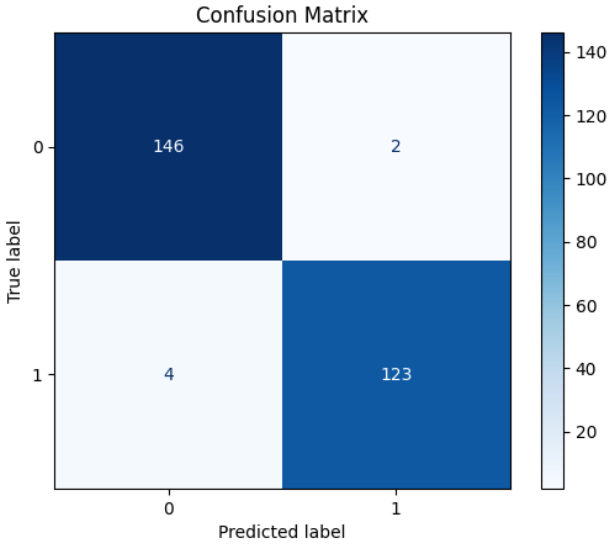
The weight evolution plot illustrates how the four feature weights are updated throughout the training process. During the initial epochs, the weights undergo relatively larger adjustments as the perceptron learns the underlying patterns in the dataset. As training progresses, the magnitude of these updates gradually decreases, with the weights fluctuating around stable values. This behaviour indicates that the perceptron has learned an effective set of parameters for binary classification while approaching convergence.

Bias Evolution



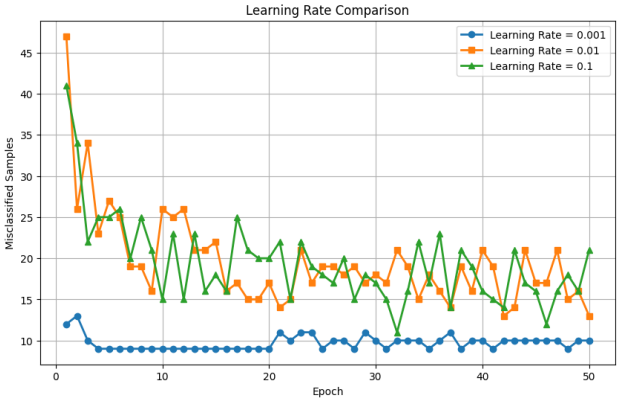
The bias evolution plot illustrates the changes in the bias term during the learning process. The bias decreases rapidly during the initial epochs as the perceptron adjusts its decision threshold. In the later stages of training, the bias fluctuates within a narrow range around a stable value, indicating that an appropriate decision threshold has been learned. The stabilization of the bias complements the convergence of the feature weights and contributes to the overall classification performance of the model.

Confusion Matrix



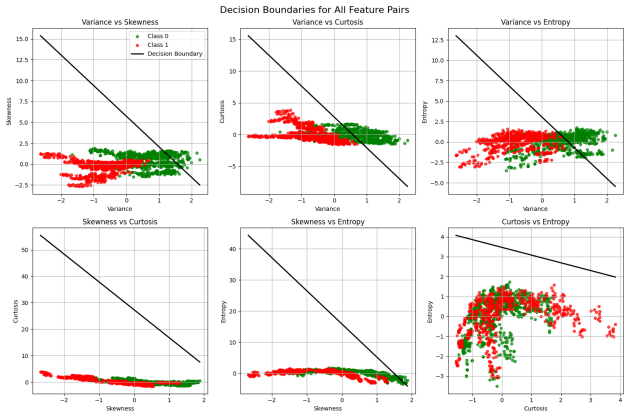
The confusion matrix summarizes the classification performance of the Single Layer Perceptron on the testing dataset. Out of 148 authentic banknotes, 146 were correctly classified as authentic, while only 2 were incorrectly classified as forged. Similarly, out of 127 forged banknotes, 123 were correctly classified, with 4 being misclassified as authentic. The low number of false positives and false negatives demonstrates the effectiveness of the perceptron in learning the decision boundary, resulting in high overall classification performance.

Learning Rate Comparison



The learning rate comparison illustrates the effect of different learning rates on the convergence behaviour of the Single Layer Perceptron. All three learning rates successfully reduce the training error without exhibiting unstable or divergent behaviour. Among the selected values, a learning rate of *0.001* achieves the lowest and most consistent training error, whereas learning rates of *0.01* and *0.1* exhibit larger fluctuations throughout the training process. These observations indicate that the choice of learning rate influences the convergence behaviour and stability of the perceptron while still enabling successful learning.

Decision Boundary



Decision boundaries were generated for all six possible combinations of the four input features to visualize the linear separation learned by the Single Layer Perceptron. Feature pairs involving *Variance* exhibit the clearest separation between authentic and forged banknotes, whereas combinations such as *Curtosis vs Entropy* show greater overlap between the classes. Although a single linear boundary cannot perfectly separate every feature pair, the generated boundaries demonstrate that the dataset is approximately linearly separable, which explains the high classification accuracy achieved by the perceptron.

9. Performance Tables

This section summarizes the training process and performance of the Single Layer Perceptron. The model hyperparameters, learned parameters, evaluation metrics, and epoch-wise learning progression are presented below.

9.1 Training Summary

Parameter	**Value**
Dataset Size	1372 Samples
Train/Test Split	80% / 20% (1097 / 275)
Learning Rate	0.01
Number of Epochs	50
Final Weights	[-0.1966, -0.2194, -0.2107, -0.0185]
Final Bias	-0.1000
Misclassified Samples (Final Epoch)	13
Accuracy	97.82%
Precision	0.9840
Recall	0.9685
F1-score	0.9762

9.2 Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	47	-0.0725	-0.0649	-0.0645	0.0002	-0.03
5	27	-0.1310	-0.1218	-0.0911	-0.0088	-0.05
10	26	-0.1317	-0.1558	-0.1260	0.0115	-0.06
20	17	-0.1564	-0.1747	-0.1552	-0.0260	-0.09
30	18	-0.1757	-0.1870	-0.1789	-0.0139	-0.10
40	21	-0.1742	-0.2153	-0.1979	-0.0149	-0.10
50	13	-0.1966	-0.2194	-0.2107	-0.0185	-0.10

10. Discussion on Additional Tasks

10.1 Comparison between Step and Sigmoid Activation Functions

The Step Activation Function returns 0 or 1 depending on whether the input value is below or above a certain threshold. So, for a perceptron, it is used for binary classification tasks. The Sigmoid Activation Function also works with binary data, but unlike the Step function, it is differentiable because its output is a continuous value that ranges between 0 and 1. The output can represent the probability of the outcome. Therefore, the main difference between the Step and Sigmoid activation functions is differentiability and continuity.

The Step Activation Function is not commonly used in most deep learning models because it is not differentiable, hence the inability to use it in backpropagation, which is used in modern machine learning algorithms. The non-linearity of most current models requires activation functions that are differentiable. Therefore, most models use Sigmoid, Tangent, or ReLU activation functions as they are differentiable.

10.2 Comparison with Scikit-learn's Perceptron

The perceptron implemented in this experiment was written from scratch, giving full control over the training process and allowing to observe how the weights, bias, and learning rule are updated. In contrast, Scikit-learn's Perceptron class is optimized and offers a variety of hyperparameters tuning and regularization options. The scratch implementation can be used as an educational tool, while the Scikit-learn's Perceptron is more appropriate for solving real problems.

10.3 Effect of Different Learning Rates

The experiment was conducted using the 0.001, 0.01 and 0.1 learning rates to determine their impact in achieving convergence of the perceptron. All the three learning rates decreased the training error and converged without instability.

It can be seen that with the learning rate of 0.001, the perceptron converged with a minimal error. The learning rates of 0.01 and 0.1 also achieved convergence but with higher fluctuations. This suggests that the learning rate affects both the speed and stability of learning in the perceptron.

10.4 XOR Problem and the Limitation of a Single Layer Perceptron

The XOR dataset is not linearly separable, which makes it impossible to separate the two classes using a single straight line. This is why the Single Layer Perceptron fails to classify all XOR samples correctly, even after many training iterations.

This suggests that it is not possible to solve some classification tasks using linearly separable models and that more complex tasks, such as those that require forming non-linearly separable decisions boundaries, require more complex networks such as Multi Layer Perceptron with several hidden layers.

10.5 Effect of Feature Normalization

Feature normalization allows for scaling all the input features into a comparable numerical range before training. Without normalizing the data, the features that have a larger numeric range may dominate the learning process, leading to unstable or slow convergence.

In this experiment, the normalization of the data allowed for higher weight updates for all the features, thus improving the convergence behavior and making the training process stable. The normalized data also helped the Single Layer Perceptron achieve a high classification accuracy.

10.6 Implementation of Logic Gates using Single Layer Perceptron

The perceptron learning algorithm was implemented from scratch in Python to model the AND, OR, XOR, and NOT logic gates. For each logic function, the perceptron was trained on the corresponding truth table. Initially, the weights and bias were set to zero, and the weights were updated using the perceptron learning rule whenever the predicted value did not match the desired value. The updated weights and bias were then printed out, and the decision boundary was plotted. The perceptron learning procedure for each gate is described below.

The AND, OR, and NOT logic gates were successfully classified by the perceptron. The decision boundary was adjusted for several epochs until there were no misclassifications. Once no misclassifications occurred during an epoch, the training stopped, indicating that the perceptron had learned the decision boundary. On the other hand, the XOR gate was not classified by the perceptron algorithm. Although the weights were updated over many epochs, the decision boundary could not separate the classifications. This is because the XOR function is not linearly separable, while the AND, OR, and NOT gates are linearly separable.

In conclusion, the perceptron was able to converge for the AND, OR, and NOT gates but not for the XOR gate. Since the perceptron can only classify linearly separable functions, it cannot converge for the XOR gate. Multi layer perceptrons (MLPs) can be used to classify non-linearly separable functions. These perceptrons have multiple layers (more than one neuron) which makes it possible to adjust the weights to classify complex patterns.

11. Conclusion

This experiment successfully demonstrated the implementation of a single layer perceptron (from scratch) for binary classification tasks. We trained and evaluated a perceptron algorithm on the Banknote Authentication dataset. We used common classification performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

The experiment revealed that perceptron demonstrated great results for the given task when features were carefully normalized and optimal hyperparameters were selected. All created figures

visualizing model performance and learning process were helpful in understanding what factors improved or reduced the accuracy of the binary classification model. Experiments with different learning rates demonstrated their impact on model convergence. The discussion of the XOR paradox illustrated the limitations of single-layer perceptrons in terms of linear separability of datasets. I got a good practical understanding of how the perceptron learning algorithm works, how it can be used for binary classification tasks, and what performance metrics need to be analyzed to evaluate such models.

12. References

1. F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

2. UCI Machine Learning Repository, "Banknote Authentication Dataset."

3. S. Raschka and V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn*. Birmingham, U.K.: Packt Publishing, 2022.

4. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

5. Scikit-learn Developers, "Scikit-learn Documentation."